

E-Commerce Product Recommendation System

**A PROJECT REPORT
for
Introduction to AI (AI101B)
Session (2024-25)**

**Submitted by
Devanshi Singhal
202410116100060
Deepanshu Ruhela
202410116100056
Dhwani Panchal
202410116100063
Aanchal
202410116100002**

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATION

**Under the Supervision of
Mr. Apoorv Jain
Assistant Professor**



**Submitted to
DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206
(MAY 2025)**

CERTIFICATE

Certified that **Devanshi Singhal (202410116100060), Deepanshu Ruhela (202410116100056), Dhvani Panchal (202410116100063), Aanchal (202410116100002)** have carried out the project work having “**Title of Report: ‘E-Commerce Product Recommendation System’ (Artificial Intelligence, AI101B)** for **Master of Computer Application** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Mr. Apoorv Jain
Assistant Professor
Department of Computer Application
KIET Group of Institutions, Ghaziabad

Dr. Akash Rajak
Dean & Professor
Department of Computer Application
KIET Group of Institutions, Ghaziabad

E-Commerce Product Recommendation System

(Devanshi Singhal, Deepanshu Ruhela, Dhwani Panchal, Aanchal)

ABSTRACT

In today's fast-paced digital economy, e-commerce platforms have become essential for consumers seeking convenience and variety. However, with an overwhelming number of products available online, users often face significant difficulty in finding items that match their preferences. Traditional search methods frequently yield irrelevant or generic results, especially when users are unsure of the exact product name or category. Furthermore, many existing recommendation systems depend heavily on user interaction history to suggest relevant items. This reliance on past data makes such systems ineffective for new users or recently added products, a limitation commonly referred to as the cold start problem.

To address these challenges, there is a need for an intelligent recommendation system that does not depend solely on user behaviour but instead leverages product information to identify similarities. This project focuses on developing a content-based recommendation system that uses Natural Language Processing (NLP) techniques to analyse and extract meaning from product metadata, such as tags and descriptions. By applying TF-IDF vectorization and cosine similarity, the system can suggest products that are contextually similar to the ones users are interested in, providing meaningful recommendations even in the absence of historical user data.

Through the integration of machine learning, clustering techniques, and a responsive web interface, the proposed system aims to enhance the user experience by simplifying product discovery, increasing engagement, and supporting more personalized shopping journeys on e-commerce platforms.

ACKNOWLEDGEMENTS

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Mr. Apoorv Jain** for his guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Akash Rajak, Professor and Dean, Department of Computer Applications**, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot in many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kinds of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Devanshi Singhal

Deepanshu Ruhela

Dhwani Panchal

Aanchal

TABLE OF CONTENTS

Certificate	ii
Abstract	iii
Acknowledgements	iv
Table of Contents	v
1 Introduction	1-3
1.1 Project Description	1
1.2 Project Scope	2
1.3 Overview	3
2 Literature Review	4-5
2.1 Introduction to Recommendation System	4
2.2 Machine Learning in Recommendation	4
2.3 Applications in E-Commerce	4-5
2.4 Dataset and Evaluation Metrics	5
3 Project Objective	6-7
4 Hardware and Software Requirements	8
5 Methodology	9-13
6 Data Analysis and Results	14-26
References	27

CHAPTER 1

INTRODUCTION

1.1 Project Description

In the digital era, e-commerce platforms have become the preferred choice for shopping, offering users the convenience of purchasing products from the comfort of their homes. However, with the rapid growth in online products and services, users often face difficulty in discovering items that truly match their preferences. To address this challenge, the **E-commerce Product Recommendation System** aims to improve user experience by providing personalized product suggestions using Machine Learning (ML) techniques.

This project implements a **content-based recommendation engine** using **Natural Language Processing (NLP)** methods, particularly **TF-IDF (Term Frequency-Inverse Document Frequency)** and **cosine similarity**. These methods analyze product attributes, such as tags or descriptions, to find and suggest items that are most similar to the one selected or searched by the user. By understanding the textual content associated with each product, the system recommends items that closely match the user's interest, even without historical user interaction data.

The core idea is to build a lightweight, flexible, and scalable recommendation system that can simulate the functionalities seen on major e-commerce websites such as Amazon, Flipkart, or Myntra. The system uses a dataset of products, including their names, brands, ratings, review counts, and image URLs. It processes this data to generate intelligent suggestions that not only enhance user satisfaction but also drive better engagement and product visibility.

Overall, the E-commerce Product Recommendation System serves as a comprehensive example of how machine learning can be practically applied to solve real-world problems in the online retail industry. It bridges the gap between raw product data and actionable user insights, offering a smart solution for both customers and businesses. This project not only showcases the power of ML in personalization but also emphasizes the importance of clean design, modular development, and user-centric features in software applications.

1.2 Project Scope

The E-commerce Product Recommendation System is designed to provide personalized product suggestions based on user input using content-based filtering techniques. The system utilizes machine learning algorithms to analyze product attributes and deliver relevant recommendations. It also supports user authentication and dynamic display of trending products. Built using Python (Flask), SQLAlchemy, and MySQL, the project demonstrates how machine learning can enhance the user experience in online shopping. The scope includes integrating product data, implementing TF-IDF and cosine similarity, and ensuring a scalable architecture for future enhancements like collaborative or hybrid recommendation models.

1.3 Project Overview

This project aims to build a content-based recommendation engine that simulates a real-world e-commerce experience. It uses a dataset of products and applies NLP techniques to recommend similar items. Users can sign up, log in, and receive recommendations by entering product names. The backend is developed using Flask and SQLAlchemy, while the frontend utilizes HTML/CSS for an interactive UI. Trending products, random prices, and images are rendered dynamically to simulate an actual shopping environment. The system demonstrates the practical application of machine learning in enhancing digital user engagement.

CHAPTER 2

Literature Review

2.1 Introduction to Recommendation Systems

Recommendation systems have become a core feature of modern e-commerce platforms such as Amazon, Flipkart, and Walmart. Their purpose is to assist users in discovering products that align with their interests, thus enhancing user experience and increasing sales. They are primarily categorized into:

- Content-Based Filtering
- Collaborative Filtering
- Hybrid Approaches

2.1.1 Collaborative Filtering

This technique is one of the most widely studied and implemented recommendation strategies. It works on the principle that users who agreed in the past tend to agree again in the future. Key references include:

- *Sarwar et al. (2001)*: Introduced item-based collaborative filtering for scalability and accuracy.
- *Su and Khoshgoftaar (2009)*: Provided an extensive survey of collaborative filtering techniques.

2.1.2. Content-Based Filtering

This method recommends products similar to those the user liked in the past, based on product attributes (e.g., category, brand, features).

- *Pazzani and Billsus (2007)*: Discussed content-based recommendation techniques using machine learning models such as decision trees and Bayesian classifiers.

2.1.3. Hybrid Systems

Hybrid recommendation systems combine multiple techniques to overcome the limitations of individual methods. For instance, cold-start problems in collaborative filtering can be mitigated using content-based models.

- *Burke (2002)*: Classified hybrid models and demonstrated their superiority in various scenarios.
- *Netflix Prize (2006–2009)*: Encouraged innovation in blending models for optimal performance.

2.2. Machine Learning in Recommendation Systems

With the rise of machine learning, algorithms such as:

- K-Nearest Neighbors (KNN)
- Matrix Factorization (e.g., SVD, NMF)
- Clustering (e.g., K-Means)
- Deep Learning (e.g., autoencoders, neural collaborative filtering) have been adopted to improve predictions.

Key works:

- *Koren et al. (2009)*: Demonstrated the effectiveness of matrix factorization models in recommender systems.
- *He et al. (2017)*: Proposed Neural Collaborative Filtering (NCF) which uses deep learning to capture complex patterns in user-item interactions.

2.3 Applications in E-Commerce

Recommendation systems directly impact:

- Conversion rates
- User engagement
- Customer retention

Studies show that personalized recommendations can contribute to over **35% of Amazon's revenue** (*McKinsey, 2013*). Other platforms like Walmart and Alibaba have integrated recommendation engines to dynamically curate user experiences.

2.4. Dataset & Evaluation Metrics

Common datasets:

- Amazon Product Reviews, Walmart Review Data

Metrics for evaluating recommendation quality include:

- Precision & Recall
- F1 Score
- Mean Squared Error (MSE)
- Mean Reciprocal Rank (MRR)

This project builds upon decades of research in recommendation algorithms, leveraging machine learning techniques to provide personalized product suggestions in e-commerce. It fits well within the current technological ecosystem where personalization is key to user satisfaction and business growth.

CHAPTER 3

Project Objective

The primary objective of the **E-commerce Product Recommendation System** is to develop a machine learning-powered solution that enhances user experience by providing intelligent, personalized product suggestions. The system aims to simulate modern e-commerce platforms by offering dynamic content, personalized results, and an interactive interface that aids users in product discovery and decision-making.

3.1. Enhance User Experience through Personalization

To implement a recommendation engine that understands product similarity based on content (tags/descriptions), thereby offering personalized suggestions to users.

Impact: Helps users find relevant products more efficiently, improving satisfaction, engagement, and overall shopping experience.

3.2. Apply Practical Machine Learning Techniques

To use content-based filtering with **TF-IDF** and **cosine similarity** to generate intelligent product recommendations.

Impact: Demonstrates real-world application of machine learning in solving practical problems in online commerce.

3.3. Design a Modular and Scalable Architecture

To build the system in a modular way, allowing for easy upgrades such as integration of collaborative filtering or hybrid models.

Impact: Ensures long-term usability and adaptability of the system in academic and industry settings.

3.4. Provide a User-Friendly and Interactive Interface

To create a clean, responsive front-end using HTML, CSS, and Jinja2 templating in Flask.

Impact: Makes the system accessible and easy to use for end-users, even those with minimal technical knowledge.

3.5. Simulate Real-World E-commerce Features

To include core e-commerce functionalities such as user login/signup, trending product listings, and mock pricing.

Impact: Provides a realistic simulation of an e-commerce platform suitable for showcasing recommendation features.

3.6. Enable Continuous Improvement and Expansion

To design the system in a way that allows for regular updates and enhancements based on testing or user feedback.

Impact: Keeps the platform relevant and effective, with potential to evolve into a more complex recommendation engine.

CHAPTER 4

Hardware and Software Requirements

4.1 Hardware Requirements

- Device: Desktop or Laptop
- Operating System: Windows 10 or later, macOS 10.13+, or any modern Linux distribution
- Processor: Intel Core i3 (minimum), Intel Core i5 or higher (recommended)
- RAM: 4 GB (minimum), 8 GB or more (recommended)
- Storage: At least 10 GB of free disk space
- Graphics: Integrated graphics (sufficient), or NVIDIA GeForce GTX 1050+ (recommended for multimedia/UI enhancements)

4.2 Software Requirements

- Web Browser: Latest version of Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari
- PDF Viewer: Adobe Acrobat Reader or equivalent for reading documentation and reports
- Text Editor/IDE: Visual Studio Code, Jupyter Notebook, or any Python-supporting IDE
- Database: MySQL Server for user authentication and data storage
- Python Libraries: Flask, SQLAlchemy, Pandas, Scikit-learn, TfidfVectorizer
- Design Tools (optional): Adobe Creative Suite, Blender, or Figma for UI/UX enhancements or design mockups

CHAPTER 5

Methodology

The proposed system employs a content-based recommendation strategy that integrates Natural Language Processing (NLP) and clustering techniques to suggest relevant products based on their textual metadata. The methodology is divided into several stages as outlined below:

5.1 Data Collection and Preprocessing

The dataset comprises product information including titles, tags, and descriptions. Preprocessing is essential to ensure consistency and quality of input data. The following NLP techniques are applied:

- Tokenization
- Lowercasing
- Stopword removal
- Lemmatization

The cleaned text forms the basis for feature extraction. The raw dataset was loaded in tab-separated format using the Pandas library. Essential columns relevant to product information were selected, such as product ID, category, brand, rating, review count, and textual metadata (product name and description). Missing values in numeric fields like ratings and review counts were imputed with zero, while textual fields were filled with empty strings to ensure processing consistency. String-based IDs were cleaned using regular expressions and converted to numeric types for indexing. Redundant and duplicate records were removed, and column names were standardized for better readability and downstream compatibility.

5.2 Feature Extraction Using Clustering

After text preprocessing, each product description is transformed into a TF-IDF vector to numerically represent the importance of terms. These vectors are then input into a K-Means clustering algorithm to group semantically similar products into clusters. Each product is assigned a cluster label, which serves as a high-level semantic feature for recommendation generation. This clustering step reduces dimensionality and noise in the vector space by organizing products into coherent groups. The cluster assignments provide an additional feature that enables category-aware recommendations, improves scalability, and assists in delivering curated suggestions. To prepare the product descriptions for content-based analysis, NLP techniques were applied using the spaCy library. Product metadata fields such as Category, Brand, and Description were cleaned by removing stop words, punctuation, and non-alphanumeric characters. Tokens were normalized to lowercase, and only semantically meaningful words were retained. These processed tokens were then aggregated into a new feature called Tags, representing a unified and enriched textual descriptor for each product. This Tags field was essential for computing textual similarity in the next stages.

5.3 Similarity Measurement

For a given product, the system computes cosine similarity between its TF-IDF vector and those of other products within the same cluster. This localized comparison ensures more contextually relevant recommendations by avoiding comparisons with unrelated items from distant clusters. Cosine similarity was used to compute the angle-based similarity between product vectors in the TF-IDF space. Products with the highest similarity scores are considered the most contextually relevant and are recommended to the user. A Python function encapsulated this logic, allowing dynamic recommendations based on real-time user input.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

5.4 Web-Based Implementation

A Flask-based web application provides the front end for user interaction.

Users can:

- Browse and search for products
- View similar product recommendations
- Register/login for personalized features
- The backend uses SQLite to manage user sessions and product data.

A web interface was built using the Flask framework to make the recommendation system interactive and user-friendly. Users can search for products, receive personalized recommendations, and interact with the application through intuitive pages such as a home page (showing top-rated items), a recommendation results page, and login/signup forms. The backend uses SQLite with SQLAlchemy ORM to manage user sessions, product data, and recommendation results.

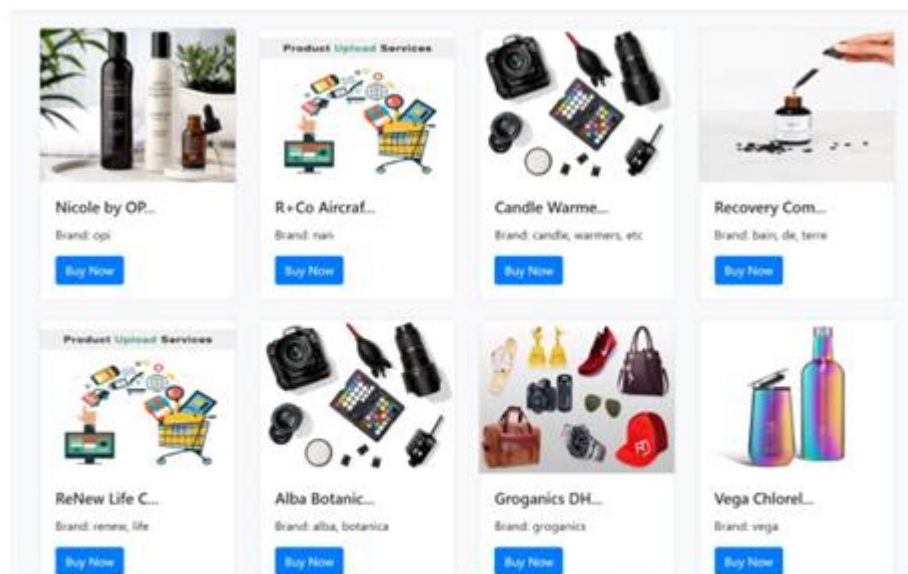


Fig. 1. Web Interface Displaying Personalized Product Recommendations

5.5 Tools and Libraries

The implementation of the e-commerce recommendation system utilized a comprehensive set of open-source libraries and frameworks. These tools were selected for their efficiency, community support, and suitability for machine learning, natural language processing, and web development tasks.

Below is a detailed overview of the key libraries and technologies employed in the project:

- **Python:** Python served as the primary language due to its versatility, readability, and vast ecosystem of libraries. It provided the foundation for data handling, machine learning, NLP processing, visualization, and backend integration.
- **Pandas:** Pandas was used extensively to load, clean, filter, and preprocess the dataset. Operations such as handling missing values, renaming columns, merging data, and generating statistical summaries were facilitated using Pandas DataFrames.
- **NumPy:** NumPy provided support for efficient numerical operations and array-based computations, especially during matrix transformations and similarity calculations.
- **Scikit-learn:** Scikit-learn played a central role in implementing:
 - TF-IDF Vectorization: To convert textual data into feature vectors.
 - Cosine Similarity: To measure similarity between product vectors and user preferences.
 - K-Means Clustering: To group similar products based on their textual or numerical features.
 - Collaborative Filtering: To compute user similarity using the cosine similarity metric on the user-item matrix.
- **spaCy:** The main purpose of using spaCy was to implement Natural Language Processing(NLP). SpaCy was used to process and clean product metadata such as descriptions, categories, and brand names. Tasks included:
 - Tokenization
 - Lowercasing
 - Stopword removal
 - Text normalizationThis helped generate meaningful and noise-free tags for content-based filtering.
- **Matplotlib and Seaborn:** These libraries were used to create:
 - Histograms of user-item interactions
 - Heatmaps to visualize the sparsity of the user-rating matrix

– Bar charts for most popular and highest-rated products

These visualizations were crucial for understanding dataset characteristics and guiding system design.

- **Flask:** Flask was used to develop the web interface, enabling real-time interaction with users. It handled: route creation for home and recommendation pages, rendering of templates, and receiving and processing user input for personalized recommendations.

5.6 Recommendation Generation and workflow

The recommendation process begins when a user inputs a product name into the web interface. To handle misspellings or partial matches, the system employs fuzzy string matching to identify the closest valid product name from the database. Once matched, the corresponding product's tags are transformed into a vector representation using TF-IDF (Term Frequency–Inverse Document Frequency). The system then computes the cosine similarity between this vector and the vectors of all other products in the dataset. Based on the similarity scores, the Top-N most similar products are selected. These recommendations are then dynamically rendered on the Flask-based user interface, providing users with personalized and contextually relevant suggestions in real time.

CHAPTER 6

Data Analysis and Results

At each phase of the project there are certain libraries are used and to implement the code the dataset is cleaned and preprocessed as follows:

6.1 Implemented Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
import os
from scipy.sparse import coo_matrix
```

Step 1: Data Loading and Preprocessing

```
train_data=pd.read_csv('marketing_sample_for_walmart_com-
walmart_com_product_review__20200701_20201231__5k_data.tsv', sep='\t')
train_data.columns
```

```
Index(['Uniq Id', 'Crawl Timestamp', 'Dataset Origin', 'Product Id',
      'Product Barcode', 'Product Company Type Source',
      'Product Brand Source', 'Product Brand Normalised Source',
      'Product Name Source', 'Match Rank', 'Match Score', 'Match Type',
      'Retailer', 'Product Category', 'Product Brand', 'Product Name',
      'Product Price', 'Sku', 'Upc', 'Product Url', 'Market',
      'Product Description', 'Product Currency',
      'Product Available Inventory', 'Product Image Url',
      'Product Model Number', 'Product Tags', 'Product Contents',
      'Product Rating', 'Product Reviews Count', 'Bsr', 'Joining Key'],
      dtype='object')
```

```
train_data = train_data[['Uniq Id', 'Product Id', 'Product Rating', 'Product Reviews Count',
'Product Category', 'Product Brand', 'Product Name', 'Product Image Url', 'Product
Description', 'Product Tags']]
train_data.head(3)
```

	Uniq Id	Product Id	Product Rating	Product Reviews Count	Product Category	Product Brand	Product Name	Product Image Url	Product Description	Product Tags
0	1705736792d82aa2f2d3caf1c07c53f4	2e17bf4acecdce67fc00f07ad62c910	NaN	NaN	Premium Beauty > Premium Makeup > Premium Nail...	OPI	OPI Infinite Shine, Nail Lacquer Nail Polish, ...	https://i5.walmartimages.com/asr/0e1f4c51-c1a4...	NaN	OPI Infinite Shine, Nail Lacquer Nail Polish, ...
1	95a9fe6f4810fc7f1244fd06784f11	076e5854a62dd283c253d6bae415af1f	NaN	NaN	Beauty > Hair Care > Hair Color > Auburn Hair ...	Nice'n Easy	Nice 'n Easy Permanent Color, 111 Natural Medi...	https://i5.walmartimages.com/asr/9c8e42e4-13a5...	Pack of 3 Pack of 3 for the UPC: 381519000201 ...	Nice 'n Easy Permanent Color, 111 Natural Medi...
2	8d4d0330178d3ed181b15a4102b287f2	8a4fe5d9c7a6ed26cc44d785a454b124	4.5	29221.0	Beauty > Hair Care > Hair Color > Permanent Ha...	Clairol	Clairol Nice N Easy Permanent Color 7/106A Nat...	https://i5.walmartimages.com/asr/e3a601c2-6a2b...	This Clairol Nice N Easy Permanent Color gives...	Clairol Nice 'N Easy Permanent Color 7/106A Na...

```
train_data['Product Tags']
```

```
0    OPI Infinite Shine, Nail Lacquer Nail Polish, ...
1    Nice 'n Easy Permanent Color, 111 Natural Medi...
2    Clairol Nice 'N Easy Permanent Color 7/106A Na...
3    Kokie Professional Matte Lipstick, Hot Berry, ...
4    Gillette TRAC II Plus Razor Blade Refills, Fit...
...
4995  Garden Mint Room Spray (Double Strength), 4 ou...
4996  Garnier Nutrisse Nourishing Hair Color Creme (...
4997  Nail File Electric Drill, 6 in 1 Professional ...
4998  Creed Love In Black Hair And Body Wash 6.8oz/2...
4999  Foundation, Wal-mart, Walmart.com
Name: Product Tags, Length: 5000, dtype: object
```

```
train_data.shape
```

Output -: (5000,10)

```
...  Uniq Id      0
     Product Id  0
     Product Rating      2806
     Product Reviews Count  1654
     Product Category      10
     Product Brand      13
     Product Name      0
     Product Image Url      0
     Product Description  1127
     Product Tags      0
     dtype: int64
```

```
train_data.isnull().sum()
```

```

# Fill missing values in 'Product Rating' with a default value (e.g., 0)
train_data['Product Rating']=train_data['Product Rating'].fillna(0)
# Fill missing values in 'Product Reviews Count' with a default value (e.g., 0)
train_data['Product Reviews Count']=train_data['Product Reviews Count'].fillna(0)
# Fill missing values in 'Product Category' with a default value (e.g., 'Unknown')
train_data['Product Category']=train_data['Product Category'].fillna("")
# Fill missing values in 'Product Brand' with a default value (e.g., 'Unknown')
train_data['Product Brand']=train_data['Product Brand'].fillna("")
# Fill missing values in 'Product Description' with an empty string
train_data['Product Description']=train_data['Product Description'].fillna("")

train_data = train_data.infer_objects(copy=False)
train_data.duplicated().sum()

```

```

# make columns shorter
# Define the mapping of current column names to shorter names
column_name_mapping = {
    'Uniq Id': 'ID',
    'Product Id': 'ProdID',
    'Product Rating': 'Rating',
    'Product Reviews Count': 'ReviewCount',
    'Product Category': 'Category',
    'Product Brand': 'Brand',
    'Product Name': 'Name',
    'Product Image Url': 'ImageURL',
    'Product Description': 'Description',
    'Product Tags': 'Tags',
    'Product Contents': 'Contents'
}
# Rename the columns using the mapping
train_data.rename(columns=column_name_mapping, inplace=True)

```

```

train_data['ID'] = train_data['ID'].str.extract(r'(\d+)').astype(float)
train_data['ProdID'] = train_data['ProdID'].str.extract(r'(\d+)').astype(float)

```

Step2 : EDA (Exploratory Data Analysis)

```

# Basic statistics
num_users = train_data['ID'].nunique()
num_items = train_data['ProdID'].nunique()
num_ratings = train_data['Rating'].nunique()
print(f"Number of unique users: {num_users}")

```

```
print(f"Number of unique items: {num_items}")
print(f"Number of unique ratings: {num_ratings}")
```

```
Number of unique users: 1721
Number of unique items: 1697
Number of unique ratings: 35
```

Pivot the DataFrame to create a heatmap

```
heatmap_data = train_data.pivot_table('ID', 'Rating')
```

```
# Create the heatmap
```

```
plt.figure(figsize=(8, 6))
```

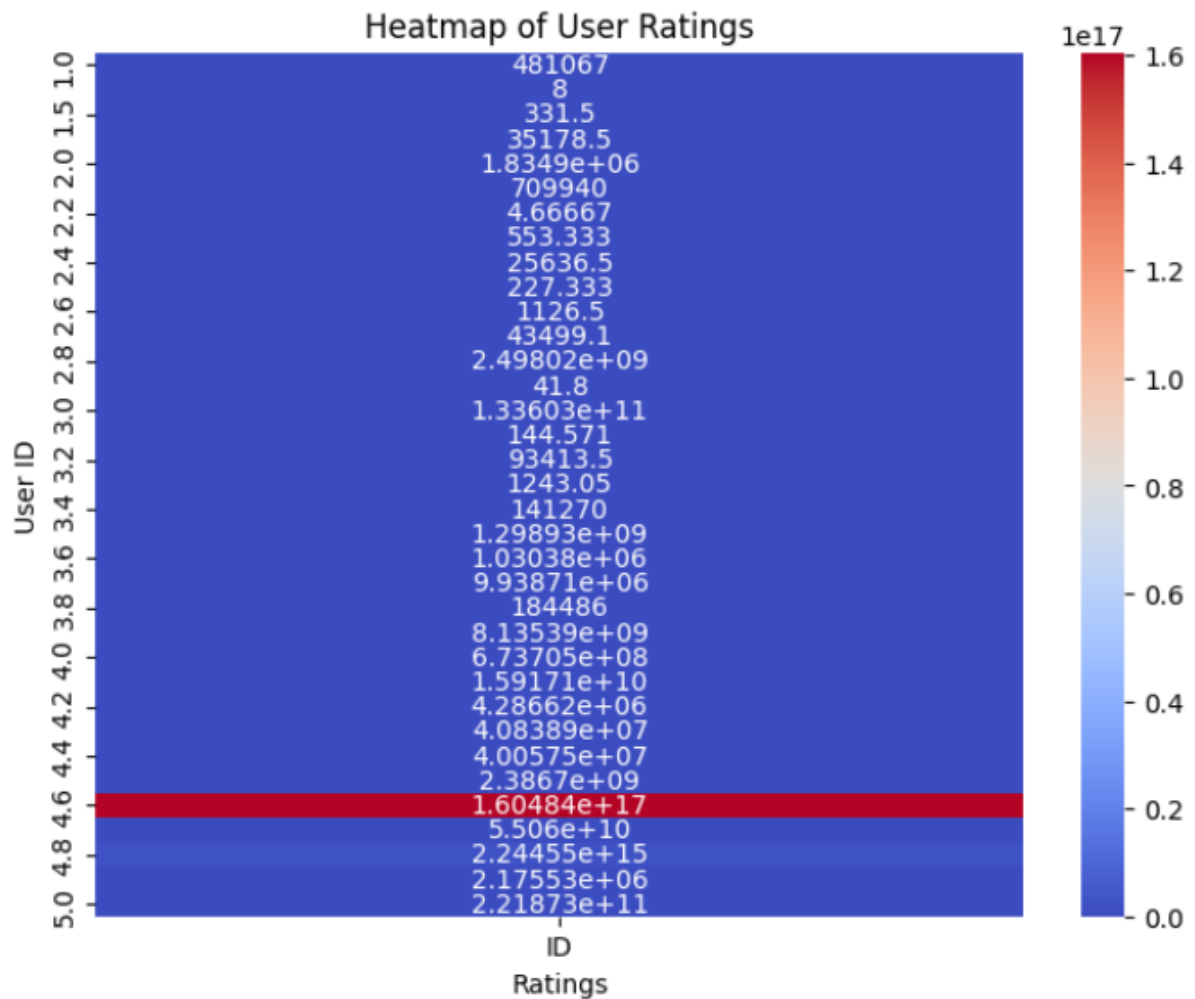
```
sns.heatmap(heatmap_data, annot=True, fmt='g', cmap='coolwarm', cbar=True)
```

```
plt.title('Heatmap of User Ratings')
```

```
plt.xlabel('Ratings')
```

```
plt.ylabel('User ID')
```

```
plt.show()
```

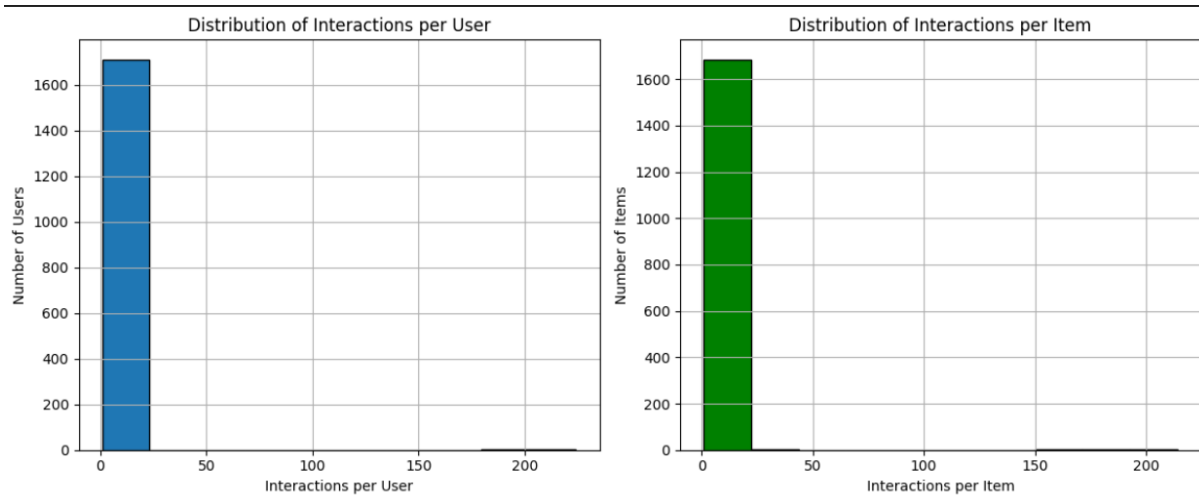


Distribution of interactions

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
train_data['ID'].value_counts().hist(bins=10, edgecolor='k')
plt.xlabel('Interactions per User')
plt.ylabel('Number of Users')
plt.title('Distribution of Interactions per User')

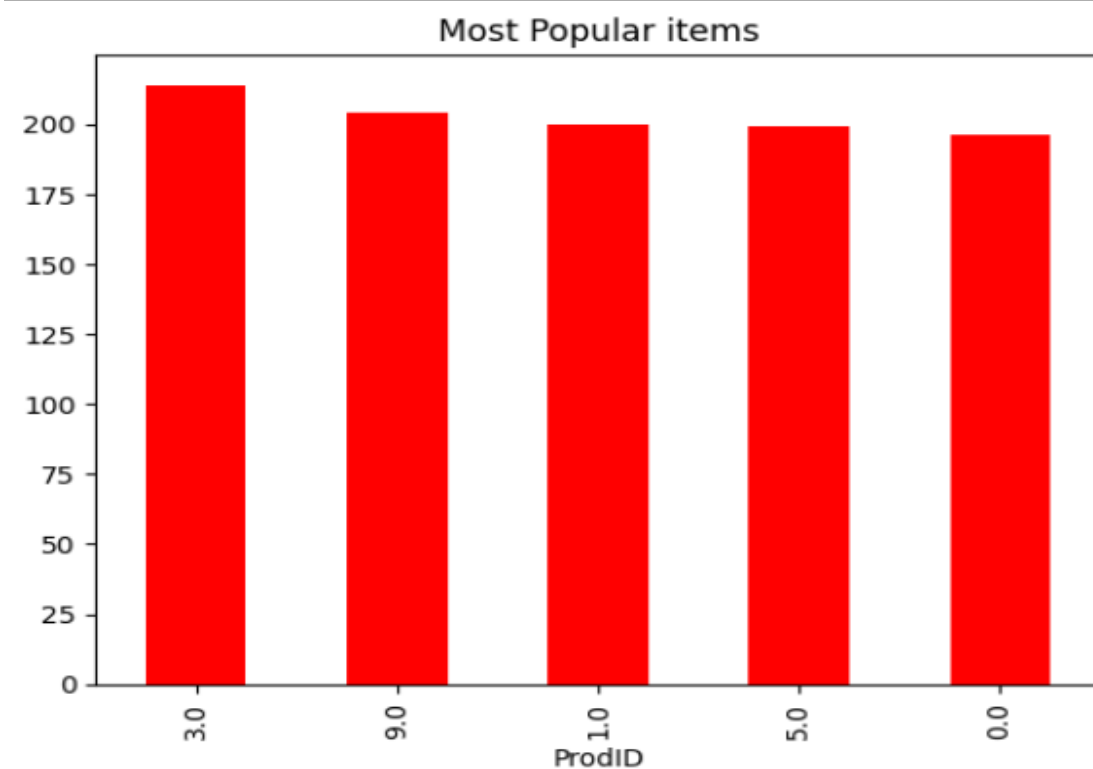
plt.subplot(1, 2, 2)
train_data['ProdID'].value_counts().hist(bins=10, edgecolor='k', color='green')
plt.xlabel('Interactions per Item')
plt.ylabel('Number of Items')
plt.title('Distribution of Interactions per Item')

plt.tight_layout()
plt.show()
```



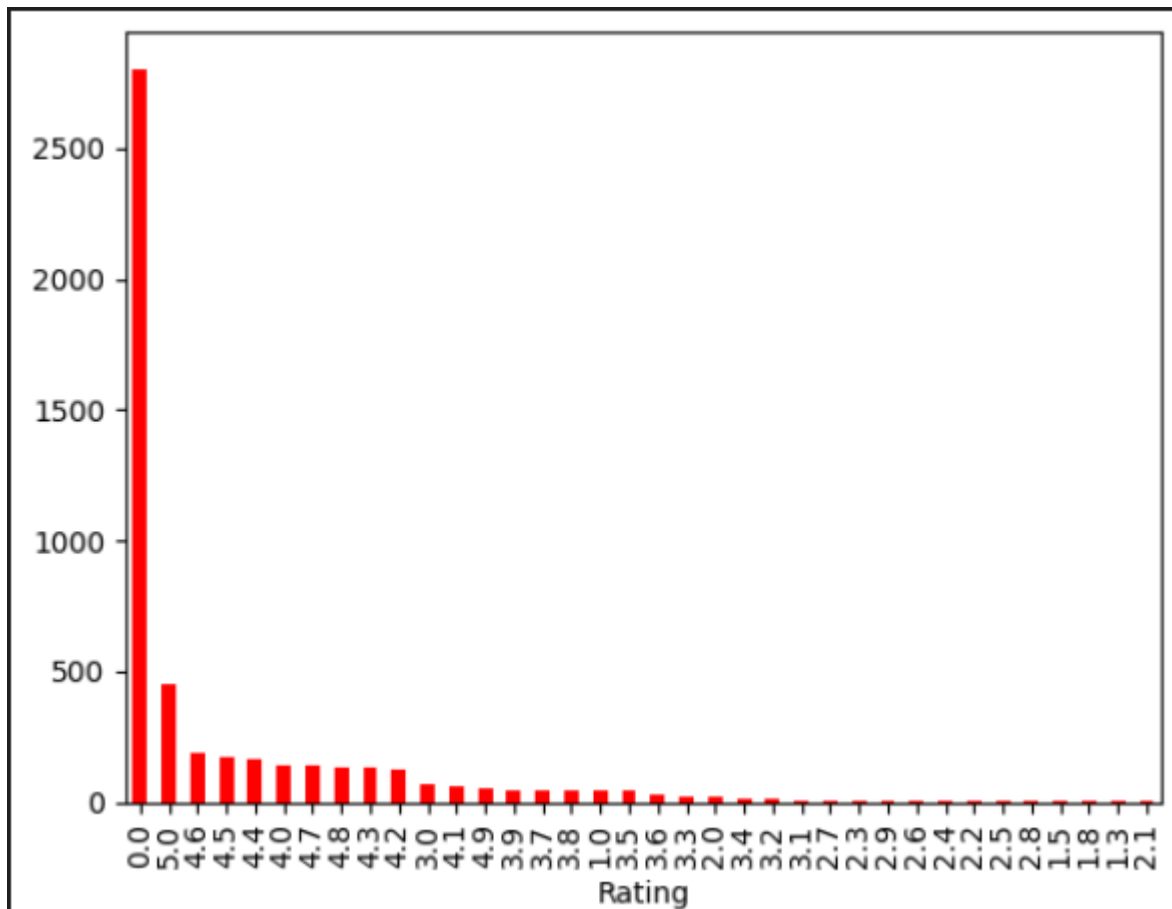
Most popular items

```
popular_items = train_data['ProdID'].value_counts().head(5)  
popular_items.plot(kind='bar',color='red')  
plt.title("Most Popular items")
```



most rated counts

```
train_data['Rating'].value_counts().plot(kind='bar',color='red')
```

Step3: Data Cleaning and Tags Creations

```
import spacy
from spacy.lang.en.stop_words import STOP_WORDS

nlp = spacy.load("en_core_web_sm")

def clean_and_extract_tags_batch(texts):
    cleaned = []
    for doc in nlp.pipe((text.lower() if isinstance(text, str) else " " for text in texts),
        batch_size=1000):
        tags = [token.text for token in doc if token.text.isalnum() and token.text not in
STOP_WORDS]
        cleaned.append(', '.join(tags))
    return cleaned

columns_to_extract_tags_from = ['Category', 'Brand', 'Description']

for column in columns_to_extract_tags_from:
    train_data[column] = clean_and_extract_tags_batch(train_data[column])
```

Concatenate the cleaned tags from all relevant columns

```
train_data['Tags'] = train_data[columns_to_extract_tags_from].apply(lambda row: ',
'.join(row), axis=1)
```

Step4: Rating Base Recommendations System

```
average_ratings =
train_data.groupby(['Name','ReviewCount','Brand','ImageURL'])['Rating'].mean().reset_index
()
```

```
top_rated_items = average_ratings.sort_values(by='Rating', ascending=False)
```

```
rating_base_recommendation = top_rated_items.head(10)
```

```
print("Rating Base Recommendation System: (Trending Products)")
rating_base_recommendation[['Name','Rating','ReviewCount','Brand','ImageURL']] =
rating_base_recommendation[['Name','Rating','ReviewCount','Brand','ImageURL']]
rating_base_recommendation
```

	Name	ReviewCount	Brand	ImageURL	Rating
4988	wnw i-shimmer eyepencil sky li	0		https://i5.walmartimages.com/asr/806e2816-e092...	0
0	\$6 Mail-In Rebate Available Braun Epilator Sil...	0		https://i5.walmartimages.com/asr/c7d78838-e8e2...	0
1	(\$100 Value) Peter Thomas Roth Thermal Therapy...	0		https://i5.walmartimages.com/asr/ecd851ef-6ca7...	0
2	(\$11 Value) Love Beauty and Planet Coconut Wat...	0		https://i5.walmartimages.com/asr/0348d82d-2a13...	0
3	(\$125 Value) Impressions Vanity Touch Pro LED ...	0		https://i5.walmartimages.com/asr/b886b154-97fd...	0
4	(\$129 Value) CHI G2 Ceramic Titanium Infused H...	0		https://i5.walmartimages.com/asr/9bcc0ed7-d62c...	0
5	(\$132 Value) Satin Smooth Hydrasonic Professio...	0		https://i5.walmartimages.com/asr/67d60a8c-32e7...	0
6	(\$14 Value) Burt's Bees Beeswax Bounty Classic ...	0		https://i5.walmartimages.com/asr/bfbb58dc-8199...	0
7	(\$14 Value) Burt's Bees Beeswax Bounty Fruit ML...	0		https://i5.walmartimages.com/asr/0cb9a07f-a2be...	0
8	(\$14 Value) NuFace Hydrating Leave-On Gel Prim...	0		https://i5.walmartimages.com/asr/b141268f-f017...	0

Step5: Content Base Recommendation system (User Preferences or Items similarities)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
tfidf_vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix_content = tfidf_vectorizer.fit_transform(train_data['Tags'])
cosine_similarities_content = cosine_similarity(tfidf_matrix_content,tfidf_matrix_content)
```

```

item_name = 'OPI Infinite Shine, Nail Lacquer Nail Polish, Bubble Bath'
item_index = train_data[train_data['Name']==item_name].index[0]

similar_items = list(enumerate(cosine_similarities_content[item_index]))

similar_items = sorted(similar_items, key=lambda x:x[1], reverse=True)
top_similar_items = similar_items[1:10]

recommended_items_indics = [x[0] for x in top_similar_items]

```

	Name	ReviewCount	Brand
4854	OPI Infinite Shine Nail Polish, Dulce De Leche...	1.0	opi
3052	OPI Nail Lacquer Nail Polish, Hawaiian Orchid	1.0	opi
4010	OPI Nail Polish, Are We There Yet?, 0.5 Fl Oz	2.0	opi
4752	OPI Infinite Shine Nail Polish, Let Love Spark...	3.0	opi
4356	OPI Nail Lacquer Nail Polish, Go with the Lava...	1.0	opi
3967	OPI Infinite Shine Nail Polish, Mini Scotland ...	1.0	opi
2952	OPI Nail Lacquer Nail Polish, Miami Beet	NaN	opi
4966	OPI Nail GelColor + Infinite Shine Polish -TAU...	NaN	opi
3252	OPI Nail Lacquer Nail Polish, Shorts Story	1.0	opi

Step6: Function To Recommend Products for Content Base

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def content_based_recommendations(train_data, item_name, top_n=10):
    # Check if the item name exists in the training data
    if item_name not in train_data['Name'].values:
        print(f"Item '{item_name}' not found in the training data.")
        return pd.DataFrame()

    # Create a TF-IDF vectorizer for item descriptions
    tfidf_vectorizer = TfidfVectorizer(stop_words='english')

    # Apply TF-IDF vectorization to item descriptions
    tfidf_matrix_content = tfidf_vectorizer.fit_transform(train_data['Tags'])

    # Calculate cosine similarity between items based on descriptions
    cosine_similarities_content = cosine_similarity(tfidf_matrix_content, tfidf_matrix_content)

```

```

# Find the index of the item
item_index = train_data[train_data['Name'] == item_name].index[0]

# Get the cosine similarity scores for the item
similar_items = list(enumerate(cosine_similarities_content[item_index]))

# Sort similar items by similarity score in descending order
similar_items = sorted(similar_items, key=lambda x: x[1], reverse=True)

# Get the top N most similar items (excluding the item itself)
top_similar_items = similar_items[1:top_n+1]

# Get the indices of the top similar items
recommended_item_indices = [x[0] for x in top_similar_items]

# Get the details of the top similar items
recommended_items_details = train_data.iloc[recommended_item_indices][['Name',
'ReviewCount', 'Brand', 'ImageURL', 'Rating']]

return recommended_items_details

```

Example: Get content-based recommendations for a specific item

```

item_name = 'OPI Infinite Shine, Nail Lacquer Nail Polish, Bubble Bath'
content_based_rec = content_based_recommendations(train_data, item_name, top_n=8)

```

content_based_rec

	Name	ReviewCount	Brand	ImageURL	Rating
4854	OPI Infinite Shine Nail Polish, Dulce De Leche...	1.0	opi	https://i5.walmartimages.com/asr/466527fe-9e5e...	5.0
3052	OPI Nail Lacquer Nail Polish, Hawaiian Orchid	1.0	opi	https://i5.walmartimages.com/asr/cef5a3ca-dfed...	4.0
4010	OPI Nail Polish, Are We There Yet?, 0.5 Fl Oz	2.0	opi	https://i5.walmartimages.com/asr/bb0c5e20-0ead...	3.0
4752	OPI Infinite Shine Nail Polish, Let Love Spark...	3.0	opi	https://i5.walmartimages.com/asr/7eef48ac-3a28...	4.0
4356	OPI Nail Lacquer Nail Polish, Go with the Lava...	1.0	opi	https://i5.walmartimages.com/asr/6cd23e3d-f3be...	5.0
3967	OPI Infinite Shine Nail Polish, Mini Scotland ...	1.0	opi	https://i5.walmartimages.com/asr/b673df9d-cc73...	0.0
2952	OPI Nail Lacquer Nail Polish, Miami Beet	NaN	opi	https://i5.walmartimages.com/asr/083f9170-da65...	0.0
4966	OPI Nail GelColor + Infinite Shine Polish -TAU...	NaN	opi	https://i5.walmartimages.com/asr/a364a6d7-0cd1...	0.0

Example: Get content-based recommendations for a specific item

```

item_name = 'Kokie Professional Matte Lipstick, Hot Berry, 0.14 fl oz'
content_based_rec = content_based_recommendations(train_data, item_name, top_n=8)

```

content_based_rec

	Name	ReviewCount	Brand	ImageURL	Rating
546	Kokie Professional Matte Lipstick, Kiss Me, 0....	NaN	Kokie Cosmetics	https://i5.walmartimages.com/asr/27dd82a2-2b9c...	NaN
3406	Kokie Professional Matte Lipstick, Firecracker...	NaN	Kokie Cosmetics	https://i5.walmartimages.com/asr/8312221b-ed22...	NaN
2873	Kokie Professional Lip Poudre Liquid Matte Liq...	7.0	Kokie Cosmetics	https://i5.walmartimages.com/asr/31c99d9b-ea11...	3.4
4050	Kokie Professional Lip Poudre Liquid Matte Liq...	7.0	Kokie Cosmetics	https://i5.walmartimages.com/asr/fdd7498c-319f...	3.4
3023	Be Matte Lipstick - Pink	2.0	City Color	https://i5.walmartimages.com/asr/4425a13e-085f...	3.0
558	Kokie Professional Nail Polish, Girls Night , ...	6.0	Kokie Cosmetics	https://i5.walmartimages.com/asr/a6f6da7a-cb11...	4.2
4920	Kokie Professional Nail Polish, Jamacan Me Blu...	6.0	Kokie Cosmetics	https://i5.walmartimages.com/asr/16fc0e60-1c66...	4.2
3620	Kokie Professional Skin Perfect HD Foundation,...	9.0	Kokie Cosmetics	https://i5.walmartimages.com/asr/2f8fcaa9-7db7...	NaN

Step7: Collaborative Filtering (User Item Similarity)

```

user_item_matrix = train_data.pivot_table(index='ID', columns='ProdID',
values='Rating',aggfunc='mean').fillna(0).astype(int)

user_similarity = cosine_similarity(user_item_matrix)

target_user_id = 4
target_user_index = user_item_matrix.index.get_loc(target_user_id)

user_similarities = user_similarity[target_user_index]

similar_user_indices = user_similarities.argsort()[::-1][1:]

recommend_items = []

for user_index in similar_user_indices:
    rated_by_similar_user = user_item_matrix.iloc[user_index]
    not_rated_by_target_user = (rated_by_similar_user==0) &
(user_item_matrix.iloc[target_user_index]==0)

    recommend_items.extend(user_item_matrix.columns[not_rated_by_target_user][1:10])

recommended_items_details =
train_data[train_data['ProdID'].isin(recommend_items)][['Name','ReviewCount','Brand','Imag
eURL','Rating']]

recommended_items_details.head(10)

```

	Name	ReviewCount	Brand	ImageURL	Rating
15	Clairol Natural Instincts Demi-Permanent Hair ...	2935.0	clairol	https://i5.walmartimages.com/asr/00a6e54a-e431...	3.7
33	DenTek Kids Fun Flossers, Removes Food & Plaqu...	3.0	dentek	https://i5.walmartimages.com/asr/de6e52eb-6e18...	0.0
61	COVERGIRL Exhibitionist Cream Lipstick, 395 Da...	713.0	covergirl	https://i5.walmartimages.com/asr/95076ec0-ffbd...	4.3
64	Neutrogena SkinClearing Oil-Free Liquid Founda...	741.0	neutrogena	https://i5.walmartimages.com/asr/fd4d78d8-310a...	4.2
69	Design Essentials Natural Coconut & Monoi Curl...	1.0	design, essentials	https://i5.walmartimages.com/asr/ff2dba1d-0c02...	5.0
78	Paul Sebastian Fine Cologne Spray, Cologne for...	28.0	paul, sebastian	https://i5.walmartimages.com/asr/03d08a07-18d7...	4.8
85	BioAstin Hawaiian Astaxanthin, Vegan, 12mg, 75 Ct	3.0	bioastin	https://i5.walmartimages.com/asr/6da9e238-b19e...	5.0
92	Bytewise Organic Moringa Leaf Powder, 12 Oz	NaN	bytewise	https://i5.walmartimages.com/asr/076f2b3f-fdc3...	0.0
94	Ag Hair Cosmetics Ultradynamics Extra-Firm Fin...	NaN	ag, hair	https://i5.walmartimages.com/asr/5d217d98-a385...	0.0
108	OPI Nail Dipping Powder Perfection Combo - Liq...	1.0	opi	https://i5.walmartimages.com/asr/ef1607ee-5bdb...	3.0

Step8: Function That Recommend Items

```
def collaborative_filtering_recommendations(train_data, target_user_id, top_n=10):
    # Create the user-item matrix
    user_item_matrix = train_data.pivot_table(index='ID', columns='ProdID', values='Rating',
aggfunc='mean').fillna(0)

    # Calculate the user similarity matrix using cosine similarity
    user_similarity = cosine_similarity(user_item_matrix)

    # Find the index of the target user in the matrix
    target_user_index = user_item_matrix.index.get_loc(target_user_id)

    # Get the similarity scores for the target user
    user_similarities = user_similarity[target_user_index]

    # Sort the users by similarity in descending order (excluding the target user)
    similar_users_indices = user_similarities.argsort()[::-1][1:]

    # Generate recommendations based on similar users
    recommended_items = []

    for user_index in similar_users_indices:
        # Get items rated by the similar user but not by the target user
        rated_by_similar_user = user_item_matrix.iloc[user_index]
        not_rated_by_target_user = (rated_by_similar_user == 0) &
        (user_item_matrix.iloc[target_user_index] == 0)

        # Extract the item IDs of recommended items

    recommended_items.extend(user_item_matrix.columns[not_rated_by_target_user][:top_n])

    # Get the details of recommended items
    recommended_items_details =
train_data[train_data['ProdID'].isin(recommended_items)][['Name', 'ReviewCount', 'Brand',
'ImageURL', 'Rating']]
```

```

return recommended_items_details.head(10)

# Example usage
target_user_id = 4
top_n = 5
collaborative_filtering_rec = collaborative_filtering_recommendations(train_data,
target_user_id)
print(f"Top {top_n} recommendations for User {target_user_id}:")
collaborative_filtering_rec

```

	Name	ReviewCount	Brand	ImageURL	Rating
61	COVERGIRL Exhibitionist Cream Lipstick, 395 Da...	713.0	COVERGIRL	https://i5.walmartimages.com/asr/95076ec0-ffbd...	4.3
85	BioAstin Hawaiian Astaxanthin, Vegan, 12mg, 75 Ct	3.0	Bioastin	https://i5.walmartimages.com/asr/6da9e238-b19e...	5.0
86	L'Oreal Paris Feria Multi-Faceted Shimmering Pe...	2144.0	L'Oreal Paris	https://i5.walmartimages.com/asr/c229026a-2b75...	3.1
108	OPI Nail Dipping Powder Perfection Combo - Liq...	1.0	OPI	https://i5.walmartimages.com/asr/ef1607ee-5bdb...	3.0
144	Covidien Curity Maternity Pad Heavy 4.338quot ...	NaN	Covidien	https://i5.walmartimages.com/asr/e4e38217-ed43...	NaN
155	Crest 3D White Brilliance Mouthwash, Alcohol F...	63.0	Crest	https://i5.walmartimages.com/asr/1fcc5525-9ae3...	NaN
174	COVERGIRL Outlast All-Day Moisturizing Lip Col...	36.0	COVERGIRL	https://i5.walmartimages.com/asr/4479896f-c6c4...	NaN
193	Revlon ColorStay Skinny Liquid Liner, 304 Gree...	70.0	Revlon	https://i5.walmartimages.com/asr/aa3b20a6-3d6d...	4.5
212	Comvita Certified UMF 20+ Manuka Honey, Raw & ...	NaN	Comvita	https://i5.walmartimages.com/asr/3cdc1498-a2ac...	NaN
241	Ahava Mens Mineral Hand Cream, 3.4 Oz	NaN	Ahava	https://i5.walmartimages.com/asr/f74e4bb7-47d3...	NaN

Step9: Hybrid Recommendations

```

# Hybrid Recommendations (Combine Content-Based and Collaborative Filtering)
def hybrid_recommendations(train_data,target_user_id, item_name, top_n=10):
    # Get content-based recommendations
    content_based_rec = content_based_recommendations(train_data,item_name, top_n)

    # Get collaborative filtering recommendations
    collaborative_filtering_rec =
collaborative_filtering_recommendations(train_data,target_user_id, top_n)

    # Merge and deduplicate the recommendations
    hybrid_rec = pd.concat([content_based_rec, collaborative_filtering_rec]).drop_duplicates()

    return hybrid_rec.head(10)

# Example usage: Get hybrid recommendations for a specific user and item
target_user_id = 10 # Change this to the user_id you want recommendations for
item_name = 'Black Radiance Perfect Tone Matte Lip Crème, Succulent Plum'

hybrid_rec = hybrid_recommendations(train_data,target_user_id, item_name, top_n=10)

print(f"Top 10 Hybrid Recommendations for User {target_user_id} and Item '{item_name}':")
hybrid_rec

```

REFERENCES

- [1] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” Proceedings of the 10th International Conference on World Wide Web, pp. 285–295, 2001.
- [2] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in Recommender Systems Handbook, Springer, pp. 73–105, 2011.
- [3] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” arXiv preprint arXiv:1301.3781, 2013.
- [4] A. S. Das, M. Datar, A. Garg, and S. Rajaram, “Google news personalization: scalable online collaborative filtering,” Proceedings of the 16th international conference on World Wide Web, pp. 271–280, 2007.
- [5] C. C. Aggarwal, Recommender Systems: The Textbook, Springer, 2016.
- [6] J. B. MacQueen, “Some methods for classification and analysis of multivariate observations,” Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, pp. 281–297, 1967. [K-Means Clustering]
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [8] M. Honnibal and I. Montani, “spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” To appear, 2017. [spaCy Library]
- [9] Python Software Foundation, “Python Language Reference,” [Online]. Available: <https://www.python.org>
- [10] Flask Documentation, “Flask Web Framework,” [Online]. Available: <https://flask.palletsprojects.com/>
- [11] M. F. Porter, “An algorithm for suffix stripping,” Program, vol. 14, no. 3, pp. 130–137, 1980. [Porter Stemmer used in NLP]
- [12] SQLite Consortium, “SQLite Documentation,” [Online]. Available: <https://www.sqlite.org>
- [13] W. Cohen, “Fast effective rule induction,” in Machine Learning Proceedings 1995, Elsevier, pp. 115–123. [On rule-based data processing, relevant to fuzzy matching]

